Defining a Class

To define a class, we use the keyword `class`, followed by a legal class name, then followed by its definition contained in braces. Legal class names follow the same rules as variable and method names, and the PSR-1 coding standard states that class names MUST be declared in StudlyCaps (also called PascalCase – each sub-word in a compound name has a leading capital letter).

Within the braces we define all of the members that the class needs. Variables are the properties, and functions are the methods. It might not be desirable to expose every member to calling code (remember how we changed the properties on an instance of `stdClass` in the previous section). Access to members from outside the class is controlled by the keywords `public` and `private`, anything declared as `private` can only be accessed by code inside the class. There is a third level of access called `protected`, which is used to control how members are inherited (covered in the next section).

Data hiding (encapsulation) is a fundamental aspect of OOP. You should only expose the properties and methods that are essential for another part of your program to make the object function correctly. Hide everything else:

```
$quoteBot = new QuoteBot();

print $quoteBot->getQuote();

class QuoteBot
{
    public array $quoteStash = [
        "Obviously, you're not a golfer." . PHP_EOL,
        "That's just, like, your opinion, man." . PHP_EOL,
        "Forget it, Donny You're out of your element." . PHP_EOL,
    ];

    public function getQuote(): string
    {
        return $this->quoteStash[rand(0,2)];
    }
}

Output:
That's just, like, your opinion, man.
```